National Textile University Faisalabad

Name:	Ayesha Khalid
Class:	BSAI-5th
Reg No:	23-NTU-CS-1248
Course name:	Embedded IoT Systems
Submitted to:	Sir Nasir Mehmood
Submission date:	19-10-2025

Question 1 — Short Questions

1. Why is volatile used for variables shared with ISRs?

- volatile is used for variables shared with ISRs so the compiler always reads their latest value (updated by ISR) directly from memory.
- Without it, the compiler may assume the variable never changes, store it in a temporary cache, and keep reading the old value instead of the one updated by the ISR, leading to incorrect results.

2. Compare hardware-timer ISR debouncing vs. delay()-based debouncing.

- **Hardware-timer ISR debouncing** uses a timer to detect switch bounce accurately without stopping other code. It is efficient and allows the CPU to keep running other tasks while the timer checks stable input.
- In contrast, **delay()-based debouncing** pauses the entire program for a fixed short period, blocking all other operations. It is easy to use but less efficient and not ideal for real-time tasks

3. What does IRAM_ATTR do, and why is it needed?

IRAM_ATTR tells the compiler to store an Interrupt Service Routine (ISR) in Internal RAM **(IRAM)** instead of Flash memory.

It is needed because code in Flash takes longer to access, and during an interrupt, speed is critical. Storing the ISR in IRAM makes it execute faster and ensures it still works even if Flash access is temporarily disabled.

4. Define LEDC channels, timers, and duty cycle.

a. LEDC Channels

LEDC channels in the ESP32 generate PWM (Pulse Width Modulation) signals. Each channel can independently control an output device such as an LED, motor, or buzzer. This allows multiple devices to operate simultaneously at different brightness or speed levels.

b. Timers

Timers in ESP32 determine the frequency of the PWM signals, means how fast the ON/OFF cycles repeat in a second.

c. Duty Cycle

Duty cycle represents the percentage of time the PWM signal is HIGH (ON) during one complete PWM cycle.

0% -- > signal always OFF (completely off)

100% --> signal always ON (full bright)

Intermediate values (like 75%) mean the device is ON for 75% of the cycle and OFF for 25% which creates a smooth analog effect.

5. Why should you avoid Serial prints or long code paths inside ISRs?

An ISR is a special function that executes when a specific hardware event(interrupt) occurs (like a button press). It interrupts the main program, runs **immediately**, and then returns control back to the main code. So ISRs should be fast and short.

We should avoid Serial prints and long code inside ISRs because ISRs must be fast and efficient. Serial printing is slow, and long or complex code can block the CPU, causing **missed interrupts,** or unexpected behavior.

6. What are the advantages of timer-based task scheduling?

Timer-based task scheduling allows tasks to run at **regular intervals**, independent of other code execution.

Advantages:

- Tasks execute periodically without delay from other code.
- Improves system efficiency by eliminating continuous polling or blocking delays.
- Enables **real-time control** for sensors, motors, and LEDs.
- Reduces CPU load and makes program design cleaner and more organized

7. Describe I²C signals SDA and SCL.

I²C (Inter-Integrated Circuit) uses two signals for communication between devices

SDA (Serial Data Line): Transfers the actual data between master(controller) and slave (peripheral) devices.

SCL (Serial Clock Line): Sends the the clock signal to make sure data is read or written at the right time. (means data is synchronized correctly).

8. What is the difference between polling and interrupt-driven input?

Polling (Continuous Checking):

- The microcontroller keeps checking if a device (like a button or sensor) needs attention.
- It wastes CPU time if nothing is happening.
- Example: Continuously asking “Is the button pressed?” in a loop.

Interrupt-Driven Input:

- The device notifies the processor only when it needs attention.
- CPU can do other tasks until the event happens, so it is more faster and efficient
- Example: Button press triggers an **interrupt**, CPu pauses it current working and immediately handles the interrupt.

9. What is contact bounce, and why must it be handled?

Contact Bounce: When a mechanical switch or button is pressed, its contacts don't close smoothly, they bounce a few times rapidly, creating multiple unwanted signals.

It must be handled because without handling, the microcontroller may read multiple presses instead of one.

So this can cause incorrect or unpredictable behavior in the system.

10. How does the LEDC peripheral improve PWM precision?

The LEDC peripheral in ESP32 is a dedicated hardware module for generating PWM signals

It improves PWM precision in such a way that

It uses hardware timers to control the PWM waveform instead of relying on the CPU.

This allows accurate timing and stable duty cycles even at high frequencies.

It reduces CPU load, so the microcontroller can handle other tasks without affecting PWM.

11. How many hardware timers are available on the ESP32?

The ESP32 has 4 independent hardware timers per timer group, and there are 2 timer groups, so in total, **8 hardware timers are available.**

Each timer can be used to generate precise(exact) timing events, PWM signals, or schedule tasks without using the CPU

12. What is a timer prescaler, and why is it used?

A timer prescaler is a feature that divides the main clock frequency before it reaches the timer(counter).

It is used because Microcontrollers have very fast clock speeds. Without a prescaler, timers would count too quickly so Prescaler slows down the timer so it can measure longer time intervals accurately.

It allows generating precise delays, PWM signals, or timed events without overflowing the timer.

Example: 16 MHz clock ÷ 8 prescaler --> timer sees 2 MHz.

13. Define duty cycle and frequency in PWM.

Duty Cycle:

- The percentage of one PWM cycle during which the signal is HIGH (ON).
- Example: 50% duty cycle → signal will ON half the time and OFF half the time.

Frequency:

- The number of complete PWM cycles that occur per second.
- Example: 1 kHz frequency → 1000 ON-OFF cycles per second.

14. How do you compute duty for a given brightness level?

Duty Value= (Brightness (%) / 100) * (Maximum count)

Example:

Desired brightness = 75%

8-bit PWM → Max Count = 2^8-1 = 255

Duty= (75/100) * 255 → 191

It means that LED at 75% brightness. 191 value is for microcontroller.

15. Contrast non-blocking vs. blocking timing.

Blocking Timing:

- The program pauses completely while waiting for a time delay.
- Example: delay(1000); stops everything for 1 second.
- **Drawback:** CPU cannot do other tasks --> inefficient.

Non-Blocking Timing:

- The program keeps running while checking how much time has passed (elapsed time).
- Example: using millis() to track duration since last action without stopping the program.
- **Advantage:** CPU can handle multiple tasks --> more efficient.

16. What resolution (bits) does LEDC support?

Resolution means precision/ no of levels available to represent a value

LEDC on ESP32 supports **1-bit to 16-bit resolution** for PWM signals.

Example:

- 8-bit → 2^8 = 256 levels (0–255) it means the LED brightness can be set to any discrete value between 0, 1, 2, 3 … up to 255.
- 10-bit → 2^{10} = 1024 levels (0–1023)
- 16-bit → 2^{16} = 65536 levels

17. Compare general-purpose hardware timers and LEDC (PWM) timers.

General-Purpose Hardware Timers:

- Used for measuring time, delays, or triggering events.
- Works internally, no direct output signal.
- Can generate interrupts on overflow.
- Example: Counting time between button presses.

LEDC (PWM) Timers:

- Used to generate PWM signals for LED, motors etc.
- Produces digital pulses with controllable duty cycle.
- Resolution can be adjusted (1–16 bits) for precise control.
- Example: LED dimming or motor speed control.

18. What is the difference between Adafruit_SSD1306 and Adafruit_GFX?

Adafruit_GFX:

- It is a graphics library that provides basic drawing functions like lines, circles, text, shapes.
- Works as a core drawing engine for many display types.
- Does not talk directly to hardware; needs a display-specific library.

Adafruit_SSD1306:

- It is a display driver library specifically for SSD1306 OLED screens.
- Handles communication with the hardware (I^2C or SPI).
- Often used with Adafruit_GFX to draw graphics on the OLED.

19. How can you optimize text rendering performance on an OLED?

Use Buffering: Draw text in memory first, then update the OLED. This reduces repeated screen refreshes.

Limit Screen Updates: Only update parts of the display that actually change, instead of refreshing the whole screen.

Use Efficient Libraries: Use optimized libraries like Adafruit_SSD1306 with Adafruit_GFX that minimize unnecessary computations.

Avoid Heavy Loops: Don't render text inside long loops or delays; update only when needed.

20. Give short specifications of your selected ESP32 board (NodeMCU-32S).

- Dual-core 32-bit CPU, 240 MHz
- 320 KB SRAM, 4 MB Flash
- Wi-Fi 802.11 b/g/n, Bluetooth 4.2 BLE
- 12-bit ADC, 8-bit DAC
- 34 GPIO pins, multiple PWM channels

Question 2 — Logical Questions

1. A 10 kHz signal has an ON time of 10 ms. What is the duty cycle? Justify with the formula.

Frequency = 10 kHz → Period (T) = 1 / 10,000 = 0.1ms
ON time = 10ms

Duty Cycle = (Ton / Period) * 100
= (10ms / 0.1ms) * 100
= 10,000%

This value is not possible because duty cycle cannot exceed 100%.
It means the ON time (10ms) is much longer than one signal period (0.1 ms).

It can be fix, either by reducing the ON time (e.g. 0.05ms) or by reducing frequency (e.g. 30Hz)

2. How many hardware interrupts and timers can be used concurrently? Justify.

Hardware Interrupts: Each GPIO can trigger its own interrupt, so multiple interrupts can work at the same time

Hardware timers: The ESP32 has 2 timer groups, each with 4 hardware timers, so total 8 timers can run concurrently.

Justification:

Each timer and interrupt has its own dedicated hardware resources (like registers and counters), so they can run independently without interfering with one another. However, if ISRs are too long or too many interrupts occur at once, the CPU may slow down, so they must be used efficiently.

3. How many PWM-driven devices can run at distinct frequencies at the same time on ESP32? Explain constraints

An ESP32 can run *up to* **4 PWM-driven devices** at distinct frequencies simultaneously, due to its 4 independent hardware timers which are shared among 16 PWM channels.

Constraints:

- Channels sharing the same timer must use the same frequency, but can have different duty cycles.
- To use different frequencies, each channel must be assigned to a different timer.
- Using too many PWM channels or very high frequencies can increase CPU load and reduce precision.

4. Compare a 30% duty cycle at 8-bit resolution and 1 kHz to a 30% duty cycle at 10-bit resolution (all else equal).

- Both signals have the same duty cycle (30%) and same frequency (1 kHz), so the LED will appear equally bright in both cases. (same ON time ratio)
- However, the **10-bit resolution** provides smoother brightness and **more precise control** because it divides the PWM signal into **1024 levels** instead of **256 levels** in 8-bit

  - **8-bit (256 levels):** 30% of 256 is: level **77**
  - **10-bit (1024 levels):** 30% of 1024 is level **307**

5. How many characters can be displayed on a 128×64 OLED at once with the minimum font size vs. the maximum font size? State assumptions.

Assumptions:

- **Display resolution:** 128 pixels wide × 64 pixels hight(tall)
- **Font type:** each character has fixed width and height
- No spacing or margins between characters or lines
- Full screen used for text only

A. Minimum Font Size (6 X 8 pixels)

- Characters per row (Columns): 128 /6 = 21.33= 21
- Rows: 64 /8 =8
- **Total characters: 21* 8 = 168**

B. Maximum Font Size (32 X 32 pixels)

- Characters per row (Columns): 128 /32 = 4
- Rows: 64 /32 = 2
- **Total characters: 4*2= 8**